

CHAP 11(2): numpy배열 데이터 처리 분석1

1차원 배열 생성 및 연산,
2차원 배열 생성 및 연산,
1차원 배열에서 데이터 추출하기,
2차원 배열에서 데이터 추출하기

[참고도서]

- 따라하며 배우는 파이썬과 데이터과학
- 오픈 데이터분석과 머신러닝

1차원 배열 생성 및 연산

- 예1) 6명의 키와 몸무게를 저장하고, BMI(체질량 지수) 계산하기.
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - BMI= 체중(kg) / 키를 제곱한 값(m^2)

```
import numpy as np
```

```
h= [1.8, 1.76, 1.69, 1.86, 1.77, 1.73]
```

```
w= [75, 70, 65, 85, 77, 68]
```

```
키= np.array(h) #키 값 배열(1차원 배열)
```

```
몸무게= np.array(w) #체중 값 배열(1차원 배열)
```

```
>>> 키   
array([ 1.8 , 1.76, 1.69, 1.86, 1.77, 1.73])  
>>> 몸무게   
array([75. , 70. , 65. , 85. , 77. , 68. ])
```

```
체질량= 몸무게 / (키**2) #연산 결과로 체질량 배열 생성
```

#체중값 배열

#키값 배열

```
>>> 체질량   
array([23.14814815 22.5981405 22.75830678  
24.56931437 24.57786715 22.72043837])
```

1차원 배열 생성 및 연산

```
print('키:', 키) # 키 배열 출력  
print('체중:', 몸무게) # 몸무게 배열 출력  
print('**BMI**')  
print(체질량) # 체질량 배열 출력
```

😊출력 화면

키: [1.8 1.76 1.69 1.86 1.77 1.73]

몸무게: [75 70 65 85 77 68]

BMI

[23.14814815 22.5981405 22.75830678 24.56931437
24.57786715 22.72043837]

1차원 배열 생성 및 연산

- 체질량을 소수점 이하 첫째자리로 정리

```
체질량= 몸무게 / (키**2)  
체질량= 체질량.round(1) # 모든 원소에 적용  
print(**BMI**)  
print(체질량)
```

😊출력 화면

```
**BMI**  
[23.1 22.6 22.8 24.6 24.6 22.7]
```

2차원 배열 생성 및 연산

- 예2) 2차원 배열에 키와 몸무게를 저장하고, BMI 계산하기.
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - BMI= 체중(kg) / 키를 제곱한 값(m²)

```
import numpy as np
h= [1.8, 1.76, 1.69, 1.86, 1.77, 1.73]
w= [75, 70, 65, 85, 77, 68]
키와몸무게= np.array( [h, w] ) #2차원 배열 생성
```

```
>>> 키와몸무게 
array([[ 1.8 ,  1.76,  1.69,  1.86,  1.77,  1.73],
       [75. , 70. , 65. , 85. , 77. , 68. ]])
>>>
```

```
체질량= 키와몸무게[1] / (키와몸무게[0]**2)
        #체중값. 1번 행      #키값. 0번 행
```

```
print('**BMI**')
print(체질량)
```

2차원 배열 생성 및 연산

```
>>> 키와몸무게   
array([[ 1.8 ,  1.76,  1.69,  1.86,  1.77,  1.73],  
       [75. , 70. , 65. , 85. , 77. , 68. ]])  
>>>
```

체질량 = 키와몸무게[1] / (키와몸무게[0]**2)
 #체중. 1번 행 #키. 0번 행

```
>>> 키와몸무게[1]   
array([75. , 70. , 65. , 85. , 77. , 68. ])  
>>>
```

#추출한 체중값.
#1차원 배열

```
>>> 키와몸무게[0]   
array([ 1.8 ,  1.76,  1.69,  1.86,  1.77,  1.73])  
>>>
```

#추출한 키값.
#1차원 배열

2차원 배열 생성 및 연산

😊출력 화면

****BMI****

[23.14814815 22.5981405 22.75830678 24.56931437
24.57786715 22.72043837])

2차원 배열 생성 및 연산

- 예3) 2차원 배열에 키와 몸무게를 저장하고, BMI 계산하기.
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - BMI= 체중(kg) / 키를 제곱한 값(m^2)

```
import numpy as np
h_w= [ [1.8, 75], [1.76, 70], [1.69, 65], [1.86, 85],
        [1.77, 77], [1.73, 68] ]
```

```
키와몸무게= np.array( h_w ) #2차원 배열 생성
체질량= 키와몸무게[:,1] / (키와몸무게[:, 0]**2)
        #체중값. 1번 열      #키값. 0번 열
```

```
print('**BMI**')
print(체질량)
```

```
>>> 키와몸무게 
array([[ 1.8 , 75. ],
       [ 1.76, 70. ],
       [ 1.69, 65. ],
       [ 1.86, 85. ],
       [ 1.77, 77. ],
       [ 1.73, 68. ]])
>>>
```


2차원 배열 생성 및 연산

```
>>> 키와몸무게   
array([[ 1.8 , 75. ],  
       [ 1.76, 70. ],  
       [ 1.69, 65. ],  
       [ 1.86, 85. ],  
       [ 1.77, 77. ],  
       [ 1.73, 68. ]])  
>>>
```

체질량= 키와몸무게[:,1] / (키와몸무게[:,0]**2)
 #체중. 1번 열 #키. 0번 열

```
>>> 키와몸무게[:,1]   
array([ 75., 70., 65., 85., 77.])  
>>>
```

```
>>> 키와몸무게[:,0]   
array([ 1.8 , 1.76, 1.69, 1.86, 1.77, 1.73])  
>>>
```

2차원 배열 생성 및 연산

😊출력 화면

****BMI****

[23.14814815 22.5981405 22.75830678 24.56931437
24.57786715 22.72043837])

1차원 배열에서 데이터 추출하기

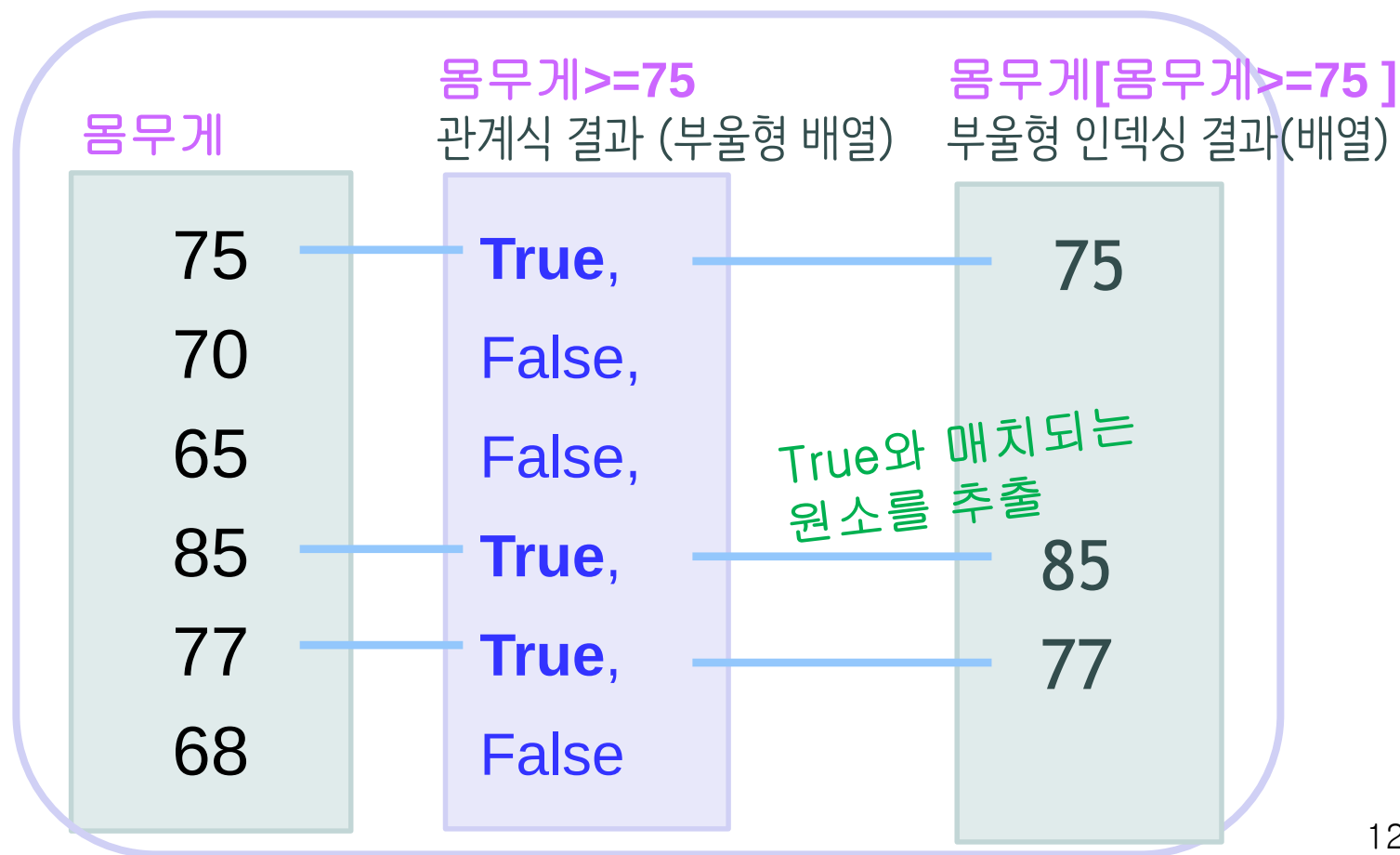
- 예1) 1차원 배열에서 , 조건을 만족하는 데이터 추출하기
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - 체중이 75kg 이상 추출하기.

```
>>> import numpy as np
>>> h= [1.8, 1.76, 1.69, 1.86, 1.77, 1.73]
>>> w= [75, 70, 65, 85, 77, 68]
>>> 키= np.array(h)          #키 값 배열. 1차원 배열
>>> 몸무게=np.array( w) #체중 값 배열
>>>
>>> 몸무게[ 몸무게>=75 ]
array([75, 85, 77])
>>>
```

```
>>> 몸무게 
array([75, 70, 65, 85, 77, 68])
>>>
```

1차원 배열에서 데이터 추출하기

- 몸무게[몸무게>=75]
 - 배열 몸무게의 원소가 75이상이면 True인 부울형 배열 생성
 - 이 부울형 배열을 인덱스로 사용, True에 대응되는 원소를 추출



2차원 배열에서 데이터 추출하기

- 예1) 2차원 배열에서, 조건을 만족하는 데이터 추출하기
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - 체중이 75kg 이상인 행(수강생 정보)을 추출하기.

```
>>> import numpy as np
>>> h_w= [ [1.8, 75], [1.76, 70], [1.69, 65], [1.86, 85],
           [1.77, 77], [1.73, 68] ]
>>> 키와몸무게= np.array(h_w)
>>> 키와몸무게[ 키와몸무게[:,1]>=75 ]
array([[ 1.8 , 75. ],
       [ 1.86, 85. ],
       [ 1.77, 77. ]])
```

- 비교: 배열에서 75이상인 원소가 추출

```
>>> 키와몸무게[ 키와몸무게>=75 ]
array([75., 85., 77.])
>>>
```

```
>>> 키와몸무게 
array([[ 1.8 , 75. ],
       [ 1.76, 70. ],
       [ 1.69, 65. ],
       [ 1.86, 85. ],
       [ 1.77, 77. ],
       [ 1.73, 68. ]])
>>>
```

2차원 배열에서 데이터 추출하기

- 키와몸무게[키와몸무게[:,1]>=75]
 - 키와몸무게[:,1]에서 75이상이면 True인 부울형 배열 생성
 - 이 부울형 배열을 인덱스로 사용, True에 대응되는 행을 추출

키와몸무게[:,1]>=75
관계식 결과 (부울형 배열)

75.	True,
70.	False,
65.	False,
85.	True,
77.	True,
68.	False

키와몸무게[키와몸무게[:,1]>=75]
부울린 인덱싱 결과(배열)

키와몸무게

[1.8,75.]
[1.76,70.]
[1.69,65.]
[1.86,85.]
[1.77,77.]
[1.73,68.]

[1.8, 75.]
[1.86, 85.]
[1.77, 77.]

True와 매치되는
행을 추출

2차원 배열에서 데이터 추출하기

- 예2) 2차원 배열에서, 조건을 만족하는 데이터 추출하기
 - 180cm-75kg, 176-70, 169-65, 186-85, 177-77, 173-68
 - 체중이 75kg 이상인 수강생 정보(행) 추출하기.

```
>>> import numpy as np
>>> h_w= [ [180,75], [176,70], [169,65], [186,85],
           [177,77], [173,68] ]
>>> 키와몸무게= np.array(h_w)
>>> 키와몸무게[ 키와몸무게[:,1]>=75]
array([[ 180, 75],
       [ 186, 85],
       [ 177, 77]])
```

```
>>> 키와몸무게
array([[ 180, 75 ],
       [ 176, 70 ],
       [ 169, 65 ],
       [ 186, 85 ],
       [ 177, 77 ],
       [ 173, 68 ]])
>>>
```

- 비교: 배열에서 75이상인 원소가 추출

```
>>> 키와몸무게[ 키와몸무게>=75]
array([180, 75, 176, 169, 186, 85, 177, 77, 173])
>>>
```

2차원 배열에서 데이터 추출하기

- 키와몸무게[키와몸무게[:,1]>=75]
 - 키와몸무게[:,1]에서 75이상이면 True인 부울형 배열 생성
 - 이 부울형 배열을 인덱스로 사용, True에 대응되는 행을 추출

키와몸무게[:,1]>=75
관계식 결과 (부울형 배열)

75	True,
70	False,
65	False,
85	True,
77	True,
68	False

키와몸무게[키와몸무게[:,1]>=75]
부울형 인덱싱 결과(배열)

키와몸무게

[180,75]
[176,70]
[169,65]
[186,85]
[177,77]
[173,68]

[180, 75]
[186, 85]
[177, 77]

True와 매치되는
행을 추출